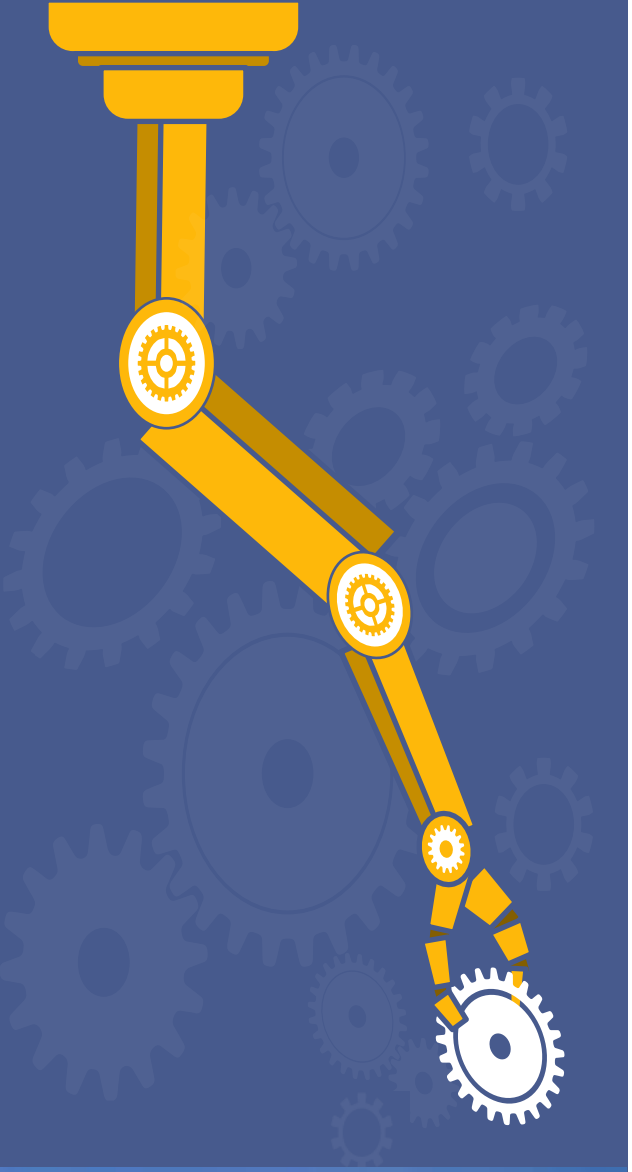


VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT, Vellore.

BCSE204L- Design and Analysis of Algorithms-

Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms

2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively

3. To synthesize efficient algorithms in various engineering design situations

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Course outcome

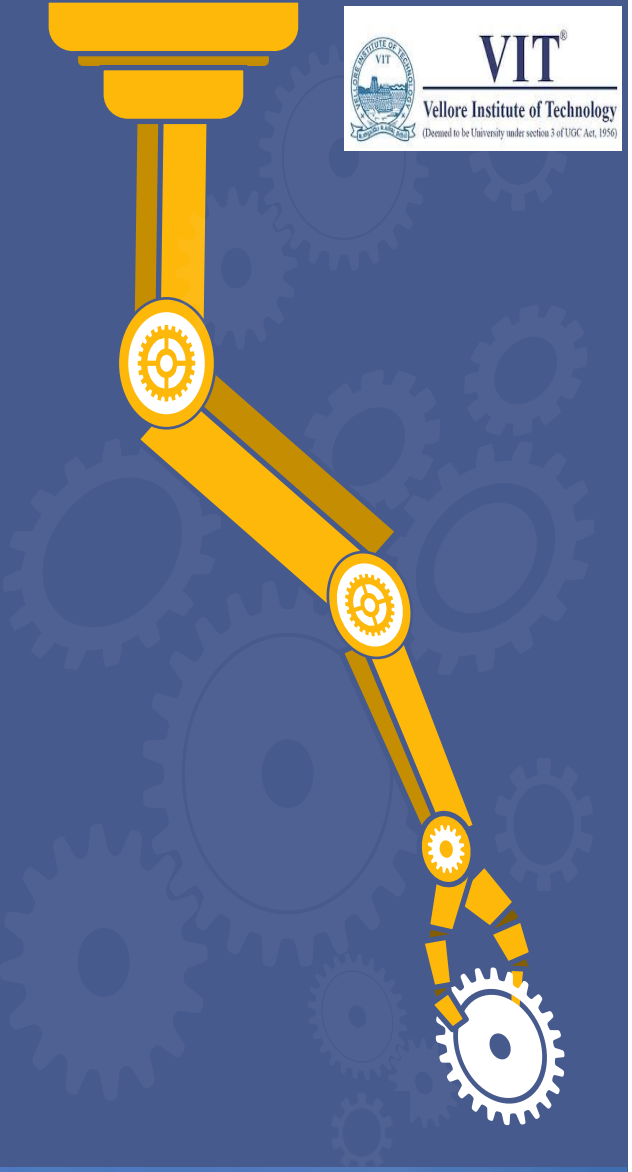
1. Apply the mathematical tools to analyse and derive the running time of the algorithms

2. Demonstrate the major algorithm design paradigms.

3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.

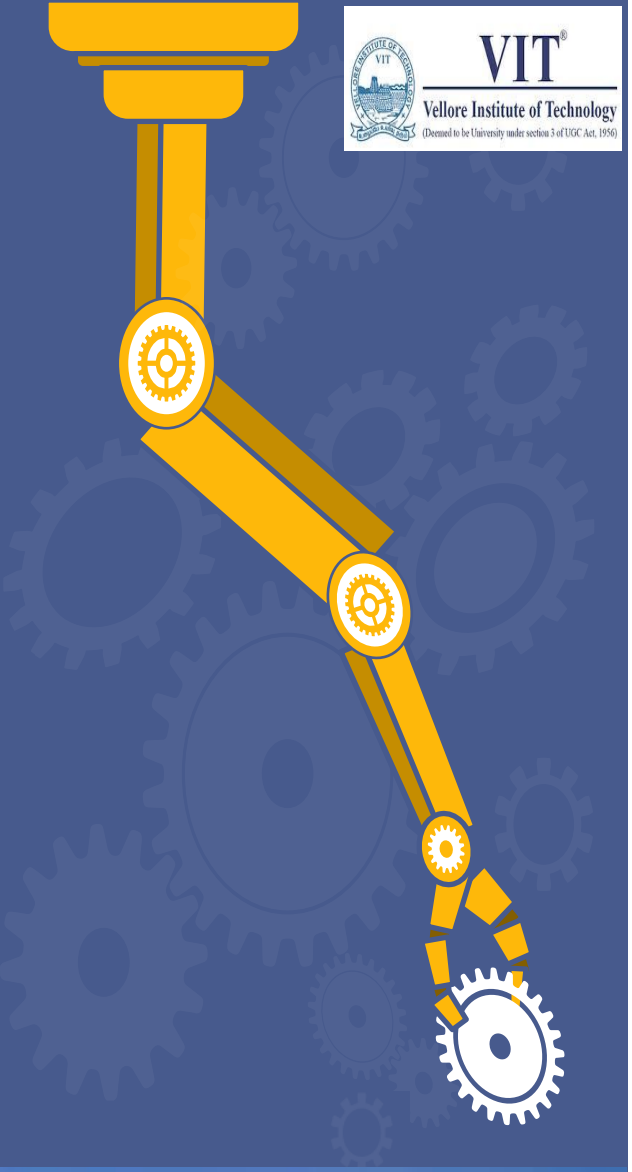
4. Articulating Randomized Algorithms.

5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	**Design Paradigms: Greedy, Divide and Conquer Techniques**	**6 hours**
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	**Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques**	**10 hours**
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	**String Matching Algorithms**	**5 hours**
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	**Graph Algorithms**	**6 hours**
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	**Geometric Algorithms**	**4 hours**
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	**Randomized algorithms**	**5 hours**
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	**Classes of Complexity and Approximation Algorithms**	**7 hours**
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	**Contemporary Issues**	**2 hours**



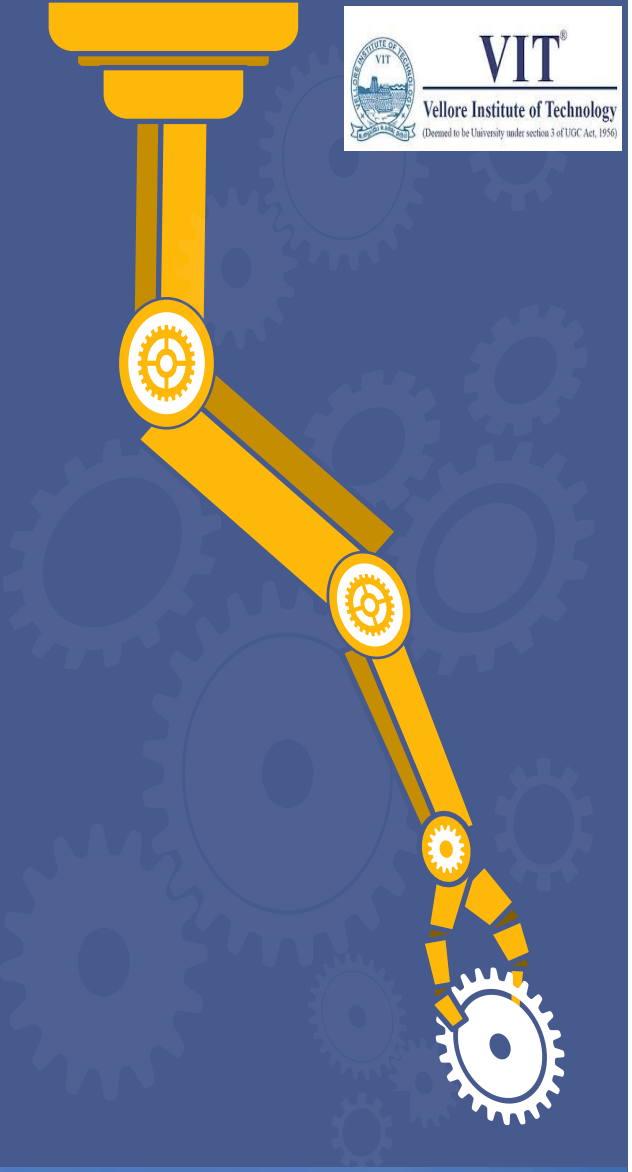
BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, ÉvaTardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press,1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, " Network Flows: Theory, Algorithms and Applications", Pearson Education, 2014.



Module 4: Graph Algorithms(6 Hours)

- Single Source Shortest Algorithm
  - **Bellman Ford Algorithm**
- All Pair Shortest Algorithm
  - Floyd - Warshall Algorithm
- Network Flows – Flow Networks
- Maximum Flow
  - Ford Fulkerson
  - Edmond Karp
  - Push Re-label Algorithm

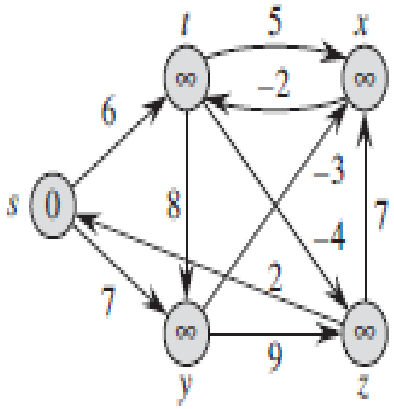
Single Source Shortest Path

- **General Objective:**
  - Given a graph and consider any one vertex as source vertex in the graph, find the shortest paths from Source vertex to all vertices in the given graph.
  - Algorithms used to Solve this problem
    - **Dijkstra's algorithm**
    - **Bellman-Ford**
- **Drawback of Dijkstra's algorithm**
  - When the graph contains negative weight edges, it doesn't works.
- **Bellman Ford Algorithm**
  - When the graph contains negative weight edges, it works well but there is no negative weight cycle.

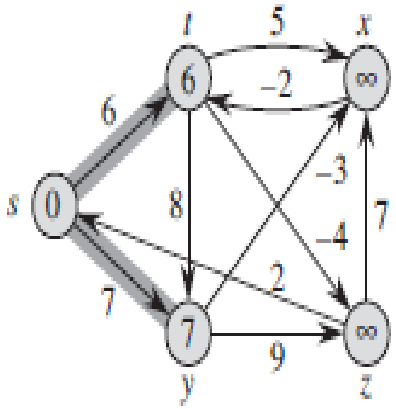
Single Source Shortest Path- Bellman Ford Algorithm

- **Bellman-Ford algorithm** solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph G = {V,E} with source s and weight function w : E->R, the Bellman-Ford algorithm returns a boolean value indicating whether or not there is a negative-weight cycle that is reachable from the source.
- If there is such a cycle, the algorithm indicates that no solution exists.
- If there is no such cycle, the algorithm produces the shortest paths and their weights.

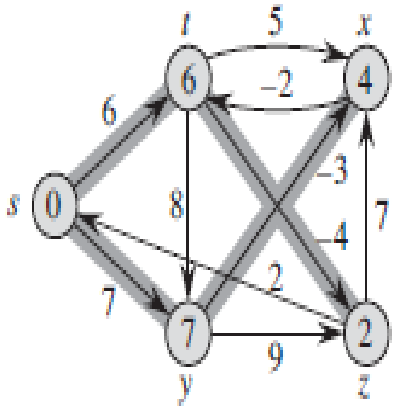
Single Source Shortest Path- Bellman Ford Algorithm



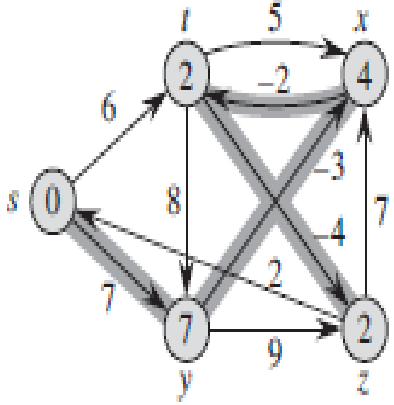
(a)



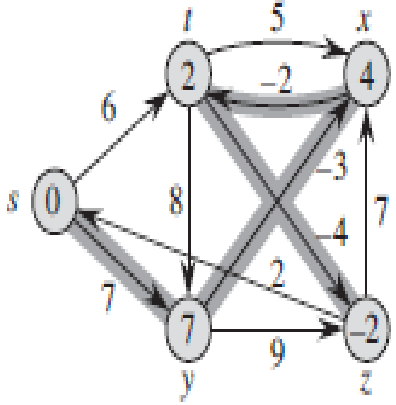
(b)



(c)



(d)



(e)

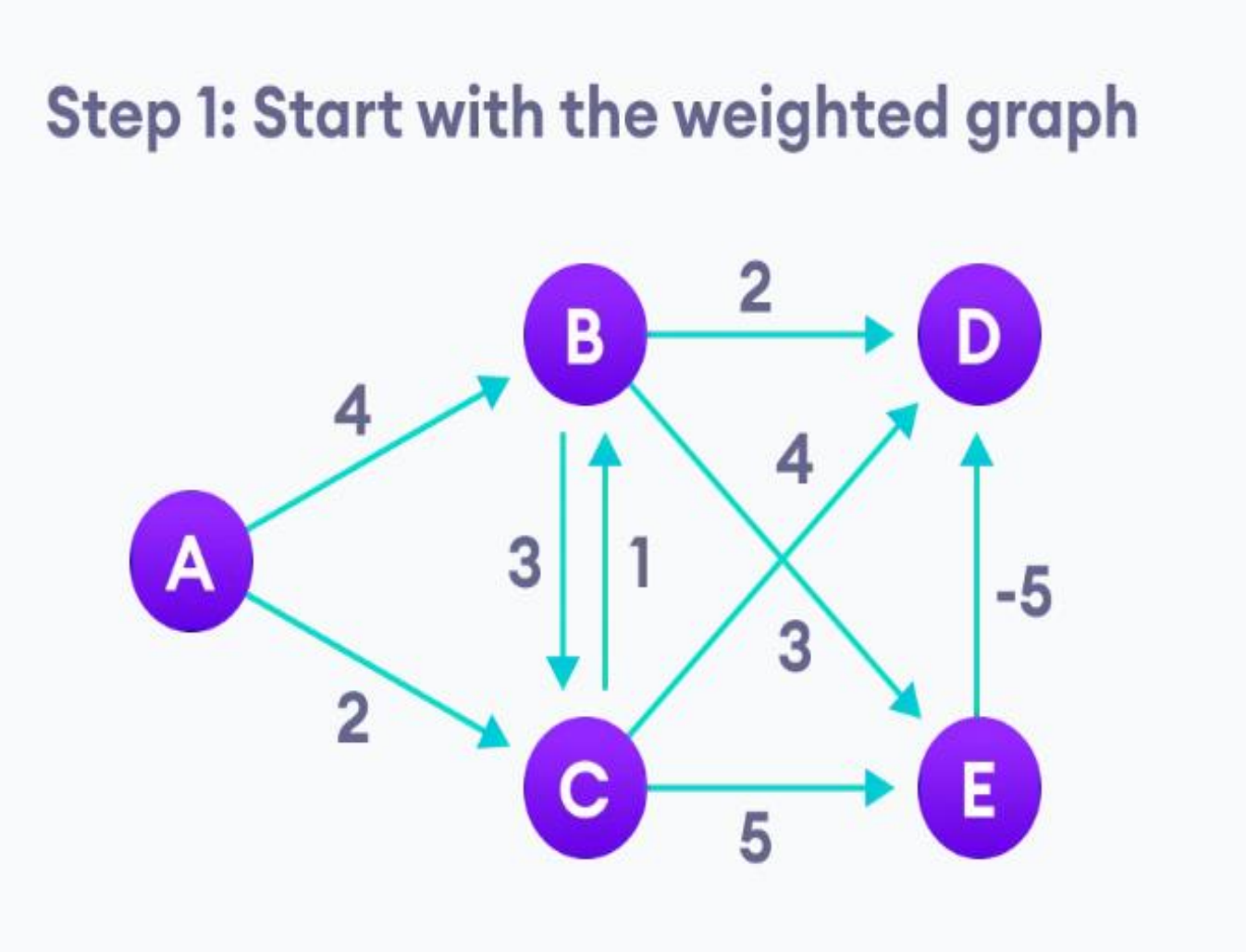
Single Source Shortest Path- Bellman Ford Algorithm

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| - 1
3      for each edge (u, v) ∈ G.E
4          RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6      if v.d > u.d + w(u, v)
7          return FALSE
8  return TRUE
```

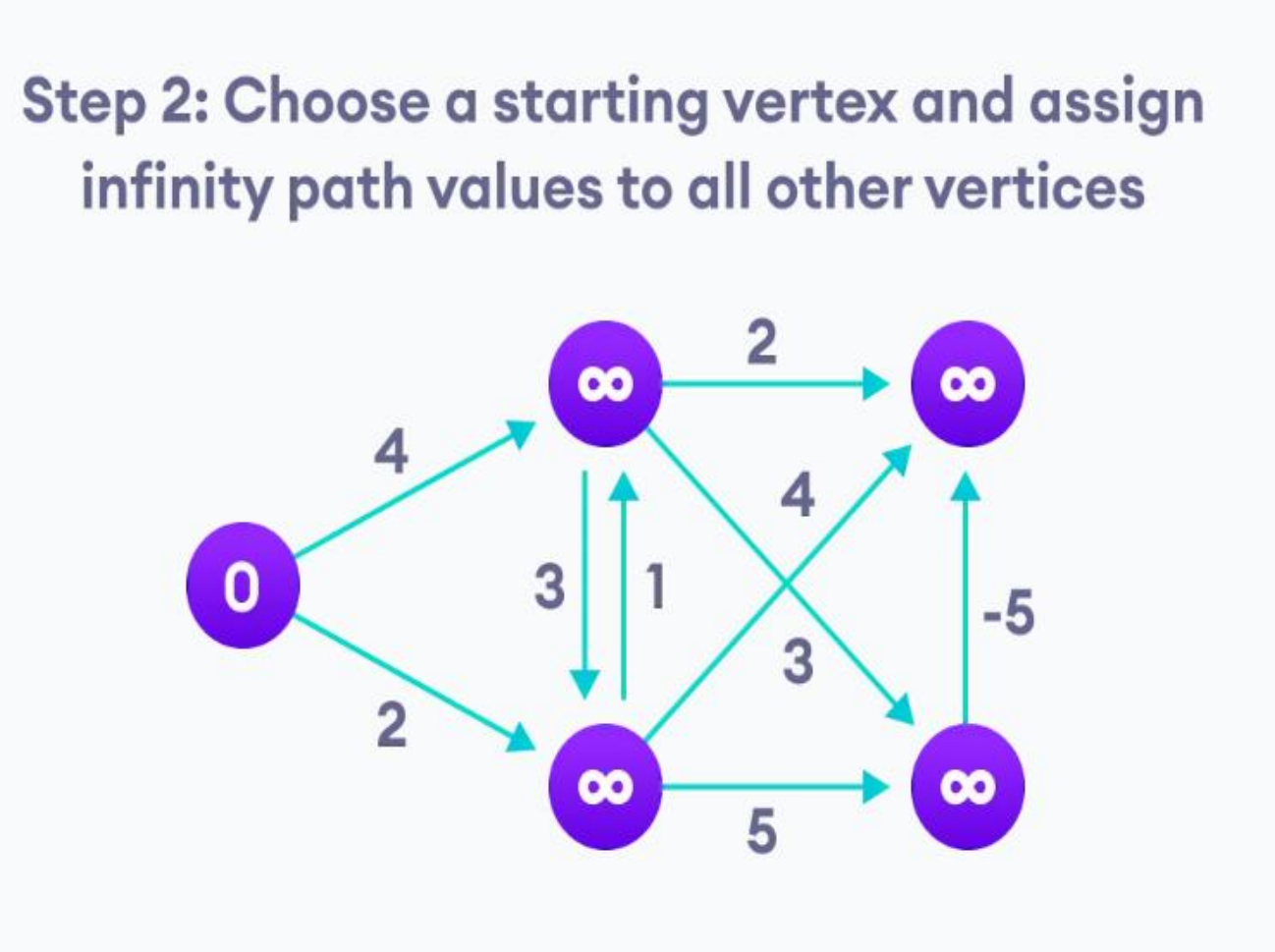
```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

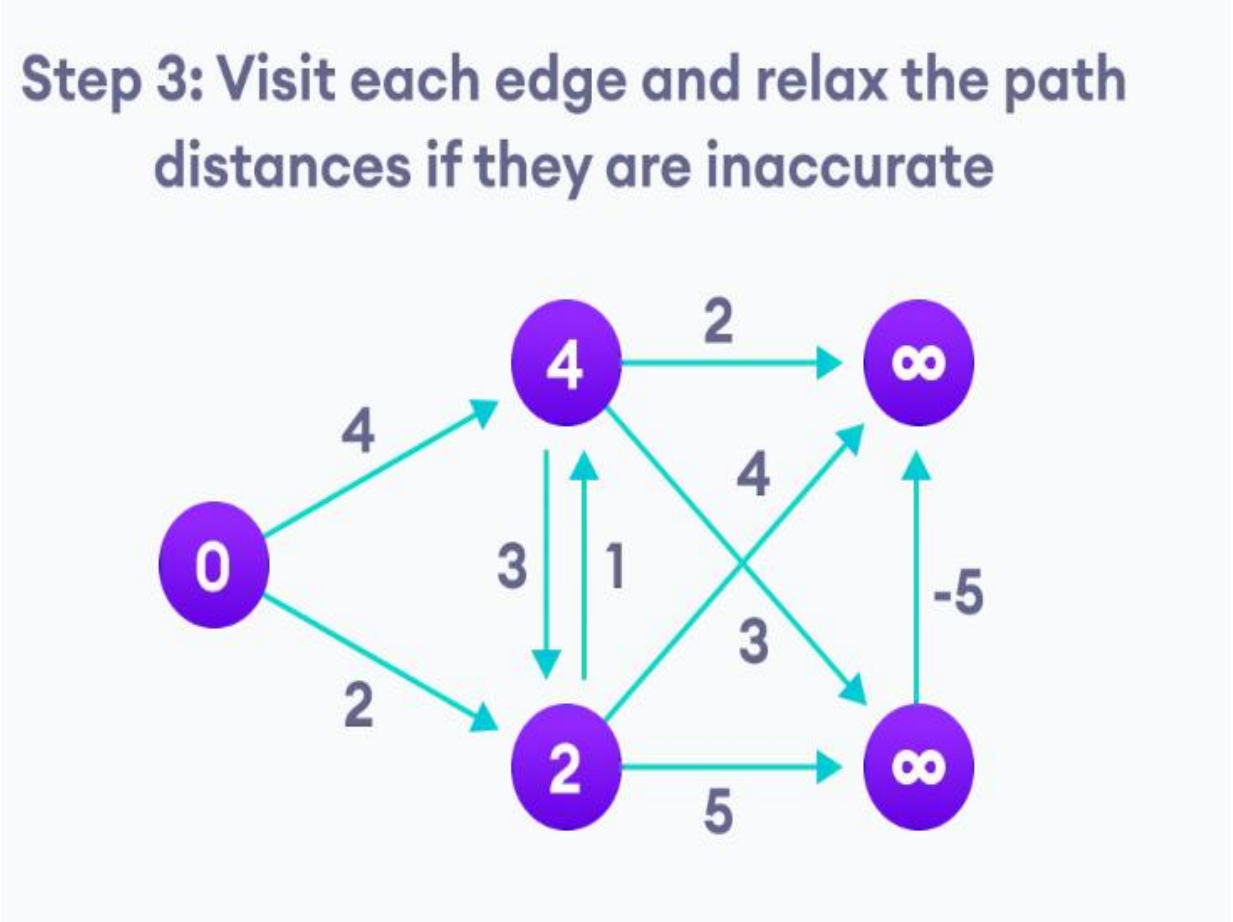
Single Source Shortest Path- Bellman Ford Algorithm- Step by step Demonstration



Single Source Shortest Path- Bellman Ford Algorithm- Step by step Demonstration

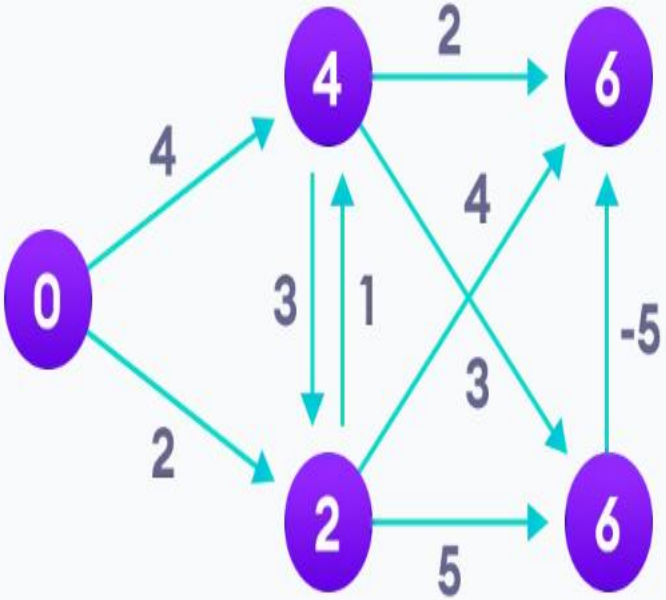


Single Source Shortest Path- Bellman Ford Algorithm- Step by step Demonstration



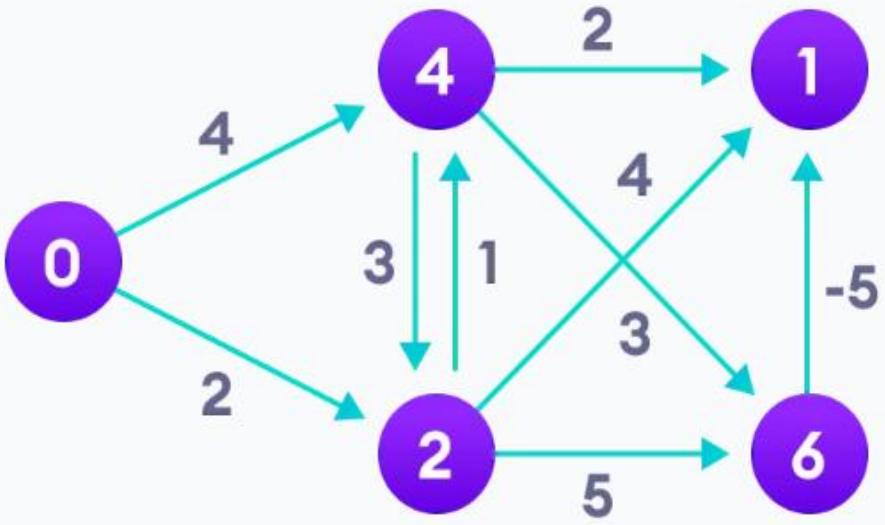
Single Source Shortest Path- Bellman Ford Algorithm- Step by step Demonstration

Step 4: We need to do this V times because in the worst case, a vertex's path length might need to be readjusted V times



Single Source Shortest Path- Bellman Ford Algorithm- Step by step Demonstration

Step 5: Notice how the vertex at the top right corner had its path length adjusted

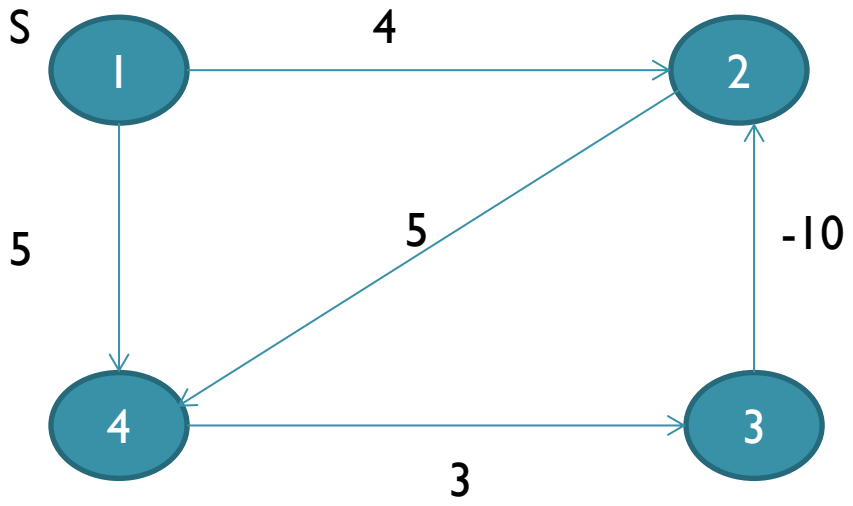


Single Source Shortest Path- Bellman Ford Algorithm- Step by step Demonstration

Step 6: After all the vertices have their path lengths, we check if a negative cycle is present

	B	C	D	E
0	∞	∞	∞	∞
0	4	2	∞	∞
0	3	2	6	6
0	3	2	1	6
0	3	2	1	6

Drawback of Bellman Ford Algorithm



Edges are (1,2), (1,4), (2,4), (3,2), (4,3)

- Bellman Ford Algorithm fails to solve these type of graph which is having negative weight cycle

Single Source Shortest Path- Bellman Ford Algorithm- Time Complexity

- Number of edges to relaxed for v-1 times
- So it is calculated as O(|E| * |v-1|)
- Generally if E is taken as number of edges as "n"
- And (v-1) iteration is taken as " n" times
- So we compute as (n*n)= **$O(n^2)$**

- **In case of Complete graph it was calculated as $O(n^3)$**